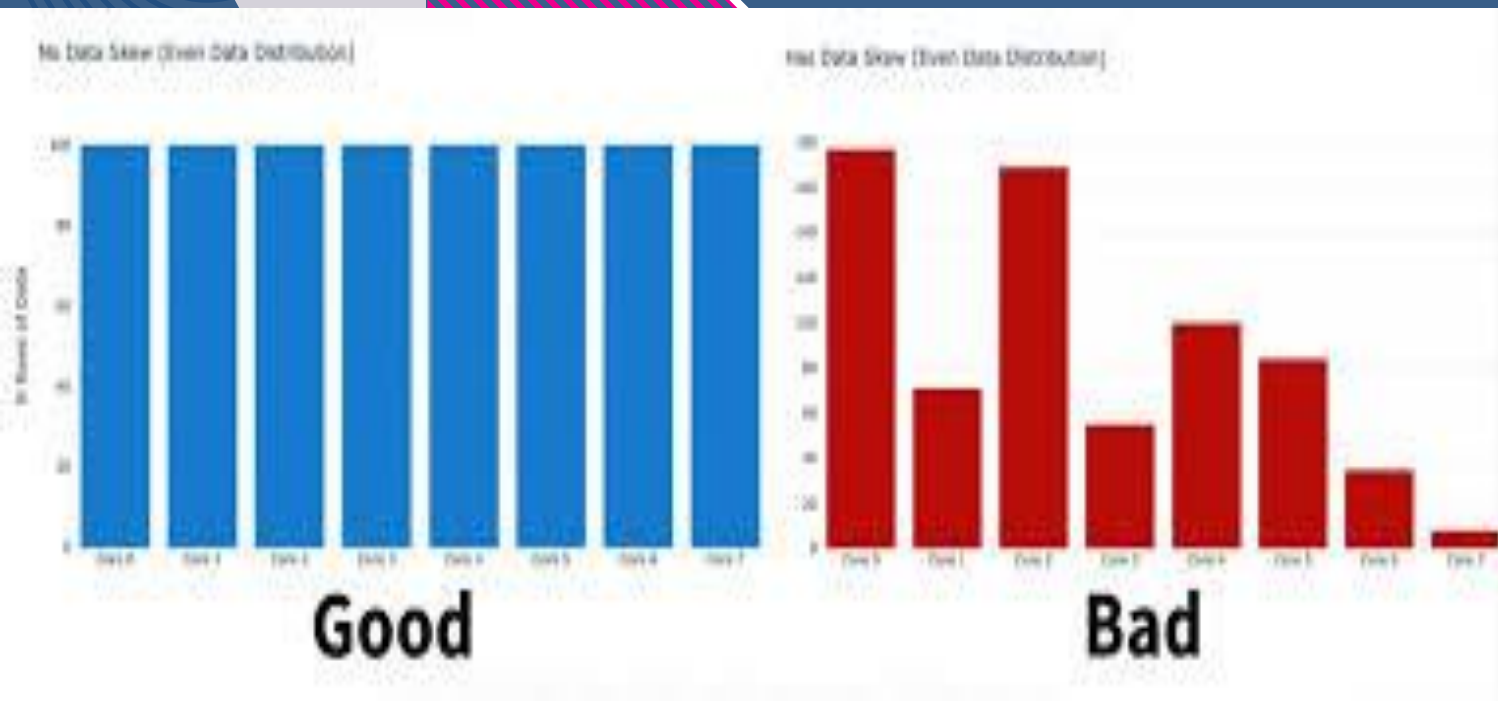


HANDLING DATA SKEW IN SPARK (JOINS & SHUFFLES)



What is Data Skew?

Spark has data loaded into memory in the form of partitions. Ideally, the data in the partitions should be uniformly distributed. Data skew is when one or some partitions have significantly more data compared to other partitions. Data-skew is usually the result of operations that require **re-partitioning** the data, mostly **join** and **grouping (GroupBy)** operations.

Data skew happens when some keys occur more frequently than others, causing uneven data distribution across partitions. This results in:

- **Straggling Tasks:** Tasks processing larger partitions take longer, causing delays and resource wastage as other tasks wait.
- **Spills to Disk and Out of Memory Errors:** Large partitions may not fit in memory, leading to disk spills and potential out-of-memory errors, further slowing down the job or causing it to fail.
- Some tasks doing way more work (**slow stages**)

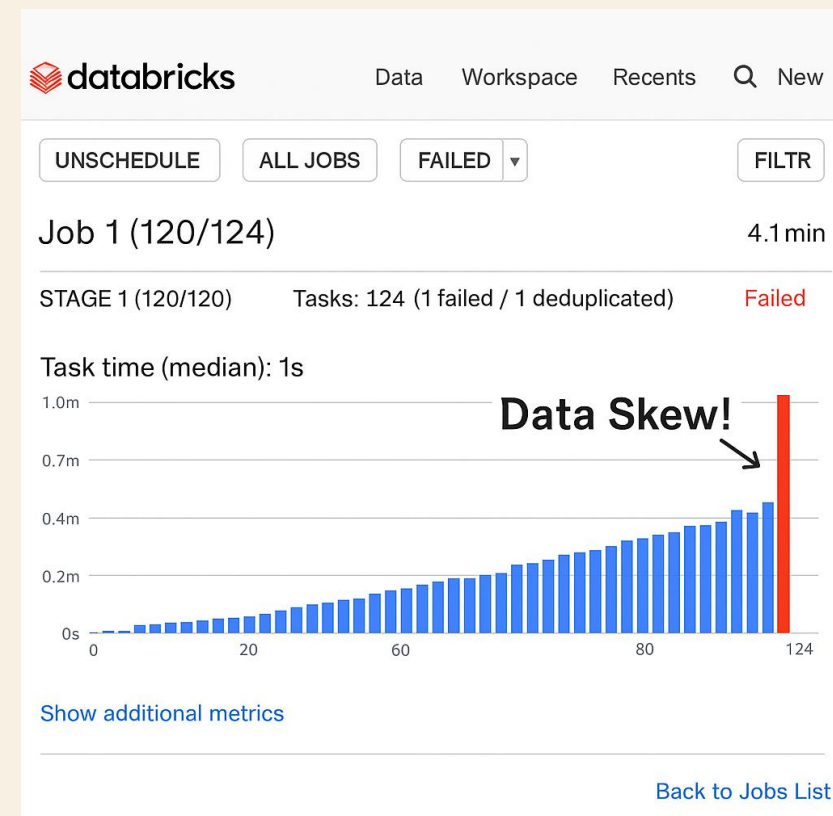
DETECTING DATA SKEW — PUTTING THE SPARK UI TO USE

© Arvind Kulkarni
#60DaysOfSpark

- 1. Disable AQE and Broadcast Joins:** Ensure Adaptive Query Execution and broadcast joins are disabled to observe skew.
- 2. Run Your Job:** Execute your Spark job and access the Spark UI
- 3. Inspect Task Duration:** Navigate to the "Stages" tab and check task durations. Significant differences between Median, 75th Percentile, and Max durations indicate skew.
- 4. Summary Metrics:** Look for tasks with much higher record counts and longer durations compared to others. These are straggling tasks causing delays.

Key Indicators of Data Skew

- ✓ **Max Duration:** If Max duration is significantly higher than the 75th percentile, skew is likely.
- ✓ **Record Counts:** Tasks processing more records than others indicate skew.
- ✓ Job hangs or fails at shuffle stage
- ✓ One or two tasks take way longer in a stage
- ✓ CPU/memory spike on few executors



1. Data Pre-processing

Data pre-processing helps minimize data skewness in Spark operations, Mitigate skewness before it hits Spark execution.

Key techniques include:

1.Data Filtering: Remove unnecessary data or outliers to create a more balanced dataset.

Example: Removing records with null or invalid keys.

2.Data Sampling: Analyze distribution using a small, random subset. Use stratified sampling for better representation across classes..

3.Data Transformation: Normalize, standardize or bin values to reduce wide key distribution.

Especially useful if the skew is due to continuous or ordinal columns.

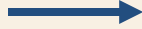
2. Data Partitioning Strategies

Ensure an even distribution before shuffle happens:

1. Hash Partitioning

Default in Spark (e.g., `groupBy`).

Hash partitioning distributes data based on the hash value of a partition key. The hash function is applied to the partition key, and the resulting hash value determines the partition in which the data will be stored.

`SELECT /*+ REPARTITION(3) */ * FROM t;`  the data from table `t` is repartitioned into 3 partitions using a hash function.

2. Range Partitioning

Range partitioning divides data into partitions based on a range of values of a partition key. Each partition holds a specific range of values, which can be useful for queries that filter data based on these ranges.

`SELECT /*+ REPARTITION_BY_RANGE(c) */ * FROM t;`

In this example, the data from table `t` is repartitioned by the range of values in column `c`.

3. Custom Partitioning

Custom partitioning allows for more complex partitioning schemes, where you can specify multiple columns and even combine different partitioning strategies. T.

3. Shuffle Join Strategies Spark uses join strategies during shuffle; choose the right one to mitigate skew:

1. Broadcast Hash Join -:

- Use when one table is small enough to fit in memory
- Avoids shuffling the larger dataset.
`df1.join(broadcast(df2), "id")`

3. Sort Merge Join

Used for large, sorted datasets.

Still subject to skew if keys aren't balanced.

5. Cartesian Join (last resort)

Joins every row with every other row.

Extremely expensive. Never use unless logically required.

2. Shuffle Hash Join

Used when both tables are large.

Partitions data by join key — leads to skew if key distribution is uneven.

4. Broadcast Nested Loop Join (expensive)

Used when no join key is present.

Note: For more details, refer back to day 14 of the series we have covered each strategy with detailed explanation, diagrams and code examples.

4. Shuffle Join Strategies Spark uses join strategies during shuffle; choose the right one to mitigate skew:

1. Broadcast Hash Join :-

- Use when one table is small enough to fit in memory
- Avoids shuffling the larger dataset.
`df1.join(broadcast(df2), "id")`

3. Sort Merge Join

Used for large, sorted datasets.

Still subject to skew if keys aren't balanced.

5. Cartesian Join (last resort)

Joins every row with every other row.

Extremely expensive. Never use unless logically required.

2. Shuffle Hash Join

Used when both tables are large.

Partitions data by join key — leads to skew if key distribution is uneven.

4. Broadcast Nested Loop Join (expensive)

Used when no join key is present.

Note: For more details, refer back to day 14 of the series we have covered each strategy with detailed explanation, diagrams and code examples.

5. Salting

Salting is a technique used to mitigate data skew in Spark by adding a random value (salt) to the keys of the data being processed. This helps distribute the data more evenly across partitions, reducing the load on any single partition.

How Salting Works:

- 1.Add Salt to Keys:** Modify the join keys by appending a random value (salt) to them.
- 2.Join on Salted Keys:** Perform the join operation on these salted keys.
- 3.Remove Salt:** After the join, remove the salt to restore the original keys.

How the Code Works

- Salting:** By adding a random value (salt) to the join keys, the data is distributed more evenly across partitions. This helps prevent any single partition from becoming a bottleneck due to data skew.
- Join Operation:** The join is performed on the salted keys, ensuring that the data is more evenly distributed during the join process.
- Result Display:** The final result of the join operation is displayed, showing the combined data from both DataFrames based on the salted keys.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit
```

```
spark = SparkSession.builder.appName("DataSkewHandling").getOrCreate()
```

```
# Example of salting
```

```
df1 = spark.range(1000000).withColumn("join_key", lit("key"))
df2 = spark.range(1000000).withColumn("join_key", lit("key"))
```

```
# Add salt to the join keys
```

```
df1 = df1.withColumn("salted_key", col("join_key") + lit("_") + (col("id") % 10))
df2 = df2.withColumn("salted_key", col("join_key") + lit("_") + (col("id") % 10))
```

```
# Perform the join on salted keys
```

```
result = df1.join(df2, df1.salted_key == df2.salted_key)
```

```
# Display the result
```

```
result.show()
```

6. Adaptive Query Execution (AQE)

Adaptive Query Execution (AQE) is a feature in Spark that dynamically optimizes query plans at runtime based on the actual data distribution and statistics. AQE helps in handling data skew by automatically adjusting the execution plan to balance the load across partitions.

How AQE Works:

1.Enable AQE: Set `spark.sql.adaptive.enabled` to `true`.

2.Dynamic Optimizations: AQE performs optimizations such as dynamically coalescing shuffle partitions, handling skewed joins, and optimizing skewed data distribution.

3.Skew Join Handling: AQE detects skewed partitions and splits them into smaller tasks to distribute the load more evenly.

7. Post-Processing Optimization

1. Repartitioning

Use `df.repartition(numPartitions, col("key"))` to manually balance partitions.

2. Bucketing (for Hive/Delta)

Predefine buckets for heavy key columns to improve join performance.

✓ Conclusion

To handle data skewness effectively:

- Preprocess data to reduce bias.
- Choose the correct join strategy.
- Apply salting or skewed key isolation for hot keys.
- Use Spark UI to observe task imbalance.
- Leverage broadcast joins where possible.